

OVP VMI/OP Function Overview

Version X-2025.09, September, 2025



Copyright and Proprietary Information Notice

© 2025 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

OVP VMI/OP Function Overview

Author:	Synopsys Inc
Version:	1.2.1
Filename:	OVP_VMI_OP_Function_Overview.doc
Project:	OVP VMI/OP Function Overview
Last Saved:	Wednesday, 12 November 2025
Keywords:	

Table of Contents

1	Introduction.....	6
2	VMI Run Time Functions	7
2.1	SIMULATION ENVIRONMENT ACCESS.....	7
2.2	PROCESSOR SIMULATION CONTROL.....	7
2.3	PROGRAM COUNTER AND CODE DICTIONARY.....	7
2.4	DICTIONARY AND BLOCK MODES	8
2.5	TIME AND CYCLE COUNTS	8
2.5.1	<i>Instruction/Cycle Counting and Interrupt</i>	<i>8</i>
2.5.2	<i>Simulated Time</i>	<i>8</i>
2.5.3	<i>Delay Estimation</i>	<i>9</i>
2.6	PROCESSOR CONNECTIONS AND REGISTERS.....	9
2.6.1	<i>Register Access.....</i>	<i>9</i>
2.6.2	<i>Bus Port Access</i>	<i>9</i>
2.6.3	<i>Net Port Access.....</i>	<i>9</i>
2.6.4	<i>FIFO Port Access</i>	<i>10</i>
2.6.5	<i>Exception Access</i>	<i>10</i>
2.6.6	<i>Connection Objects</i>	<i>10</i>
2.7	MEMORY OPERATIONS.....	10
2.7.1	<i>Generic Load/Store</i>	<i>10</i>
2.7.2	<i>Memory Callbacks.....</i>	<i>11</i>
2.7.3	<i>Memory Manipulation</i>	<i>11</i>
2.7.4	<i>ASID Memory Management</i>	<i>11</i>
2.8	FLOATING POINT	12
2.8.1	<i>Floating-Point Operation Control.....</i>	<i>12</i>
2.8.2	<i>Floating-Point Operations</i>	<i>12</i>
2.9	SMP PROCESSOR HIERARCHY.....	12
2.10	OBJECT FILE ACCESS	12
2.10.1	<i>Translation between Object File Addresses and Names.....</i>	<i>13</i>
2.10.2	<i>Add and Access Object Files</i>	<i>13</i>
2.10.3	<i>Query Object File Symbols.....</i>	<i>13</i>
2.10.4	<i>Query Object File File/Line</i>	<i>13</i>
2.11	RANGE TABLE HASH.....	13
2.12	SHARED DATA	14
2.13	NOTIFIERS	14
2.13.1	<i>Register Update Notifiers</i>	<i>14</i>
2.13.2	<i>Branch Reason Notifiers.....</i>	<i>14</i>
2.14	SAVE/RESTORE SUPPORT	14
2.15	DEBUG VIEW SUPPORT	14
2.16	INSTRUCTION ATTRIBUTES.....	15
2.17	SHARED OBJECT / DYNAMIC LINKED LIBRARY LOADING	15
2.18	COMMAND INTERPRETER	15
2.19	DEBUGGER INTEGRATION	15
2.20	TRACE INTEGRATION	16
2.21	DOCUMENTATION	16
2.22	MESSAGES.....	16
2.23	HTTP INTERFACE	16
2.24	LICENSING.....	16
2.25	ENCAPSULATED MODEL.....	16
3	OP Functions	17
3.1	SIMULATION.....	17
3.1.1	<i>Session Control.....</i>	<i>17</i>

3.1.2	<i>Execution Control</i>	17
3.1.3	<i>Environment Access</i>	17
3.1.4	<i>Module Simulation</i>	18
3.2	PROGRAM COUNTER AND CODE DICTIONARY.....	18
3.3	TIME AND CYCLE COUNTS.....	18
3.3.1	<i>Instruction/Cycle Counting and Interrupt</i>	18
3.3.2	<i>Simulated Time</i>	18
3.3.3	<i>Delay Estimation</i>	18
3.4	PLATFORM COMPONENT CREATION, CONNECTION AND QUERY.....	19
3.4.1	<i>Bus Bridges</i>	19
3.4.2	<i>Buses</i>	19
3.4.3	<i>Bus Slaves</i>	19
3.4.4	<i>Bus Ports</i>	19
3.4.5	<i>Bus Port Connections</i>	19
3.4.6	<i>Extensions</i>	20
3.4.7	<i>FIFOs</i>	20
3.4.8	<i>FIFO Ports</i>	20
3.4.9	<i>FIFO Port Connections</i>	20
3.4.10	<i>MMCs</i>	20
3.4.11	<i>Memory-Mapped Registers</i>	20
3.4.12	<i>Memories</i>	21
3.4.13	<i>Modules</i>	21
3.4.14	<i>Nets</i>	21
3.4.15	<i>Net Ports</i>	21
3.4.16	<i>Net Port Connections</i>	22
3.4.17	<i>Objects</i>	22
3.4.18	<i>Packet Nets</i>	22
3.4.19	<i>Packet Net Port Connections</i>	23
3.4.20	<i>Peripherals</i>	23
3.4.21	<i>Processors</i>	23
3.4.22	<i>VLNV</i>	24
3.5	MEMORY OPERATIONS.....	24
3.5.1	<i>Generic Load/Store</i>	24
3.5.2	<i>Memory Callbacks</i>	24
3.5.3	<i>Memory Manipulation</i>	25
3.6	SMP PROCESSOR HIERARCHY.....	25
3.7	APPLICATION FILE ACCESS.....	25
3.8	SHARED DATA.....	26
3.9	PROCESSOR REGISTERS, EXCEPTIONS AND MODES.....	26
3.9.1	<i>Processor Registers</i>	26
3.9.2	<i>Mode Access</i>	26
3.9.3	<i>Exception Access</i>	27
3.10	PARAMETERS.....	27
3.10.1	<i>Formal Parameters</i>	27
3.10.2	<i>Actual Parameters</i>	27
3.11	SAVE/RESTORE SUPPORT.....	28
3.12	INSTRUCTION ATTRIBUTES.....	28
3.13	COMMAND INTERPRETER.....	29
3.14	DEBUGGER INTEGRATION.....	29
3.15	BREAKPOINTS.....	29
3.16	WATCHPOINTS.....	29
3.17	TRIGGERS.....	30
3.18	TRACE INTEGRATION.....	30
3.19	DOCUMENTATION.....	30
3.20	MESSAGES.....	31
3.21	HTTP INTERFACE.....	31

1 Introduction

This document provides a high-level overview of the functional groups implemented in the VMI run time and OP interfaces. The intention is not to describe any functions in detail, but to provide a starting point about where to look for functions required to implement a particular task.

The VMI and OP APIs are intended to perform different tasks. The VMI interface is primarily intended for *modeling of processors*. It therefore contains functions for describing instructions to the JIT translation engine, efficient implementation of memory translation schemes, description of processor registers and so on. The OP API is primarily intended for *description of platform interconnect* and *simulation control*. It therefore contains functions for instantiation of components, executing processors and so on. The great majority of tasks that are done when modeling a processor are inappropriate for use in a platform/simulation context and vice-versa, so they are placed in different APIs to guard against misuse.

There is some overlap in the two interfaces when they are used for *tool development*. Tools are sometimes implemented as intercept libraries on processor objects and sometimes in a simulation harness written using the OP API. Therefore, some functions in these two APIs perform similar purposes: this document indicates which functions fall into such groups.

For detailed information on the VMI Run Time function API, consult the *VMI Run Time Function Reference Manual*. For detailed information on the OP API, consult the Doxygen documentation tree in the Imperas installation.

2 VMI Run Time Functions

2.1 *Simulation Environment Access*

These functions are used to access simulation environment features:

```
vmirtSuppressStdout  
vmirtPlatformName
```

2.2 *Processor Simulation Control*

These functions control inspection and execution of a processor:

vmirtGetCurrentProcessor	opProcessorCurrent
vmirtCPUId	
vmirtGetProcessorForCPUId	
vmirtProcessorFlags	
vmirtProcessorName	
vmirtSetProcessorName	
vmirtProcessorVariant	opProcessorVariant
vmirtSetProcessorVariant	
vmirtProcessorType	
vmirtProcessorStringAttribute	
vmirtProcessorBoolAttribute	
vmirtProcessorUns32Attribute	
vmirtProcessorUns64Attribute	
vmirtProcessorFlt64Attribute	
vmirtGetCurrentMode	opProcessorModeCurrent
vmirtGetNextMode	opProcessorModeNext
vmirtYield	
vmirtHalt	
vmirtInterrupt	opInterrupt
vmirtYieldControl	opProcessorYield
vmirtExit	opProcessorExit
vmirtFinish	opProcessorFinish
vmirtStop	opInterruptRSP
vmirtAtomic	
vmirtBlock	
vmirtAbortRepeat	
vmirtIsHalted	
vmirtRestartNext	
vmirtRestartNow	
vmirtDoSynchronousInterrupt	

For related OP functions, see section 3.4.21.

2.3 *Program Counter and Code Dictionary*

These functions handle access to simulated program counter and invalidation of code dictionary:

vmirtGetPC	opProcessorPC
vmirtGetPCDS	opProcessorPCDS
vmirtSetPC	opProcessorPCSet
vmirtSetPCDS	
vmirtSetPCException	

```
vmirtSetPCFlushTarget
vmirtSetPCFlushDict
vmirtFlushTarget
vmirtFlushTargetMode
vmirtFlushTargetModeTagged
vmirtFlushDict
vmirtFlushAllDicts
vmirtAddPCCallback
vmirtRemovePCCallback
vmirtUpdatePCCallbackCondition
vmirtGetPCCallbackCondition
```

For related OP functions, see section 3.2.

2.4 Dictionary and Block Modes

These functions handle mode-specific JIT code:

```
vmirtGetMode
vmirtSetMode
vmirtGetBlockMask
vmirtSetBlockMask
vmirtSetBlockMask32
vmirtSetBlockMask64
```

2.5 Time and Cycle Counts

2.5.1 Instruction/Cycle Counting and Interrupt

These functions handle instruction and cycle counts:

```
vmirtGetProcessorIPS
vmirtGetICount
vmirtGetExecutedICount
vmirtSetICountInterrupt
vmirtClearICountInterrupt
vmirtCreateModelTimer
vmirtCreateMonotonicModelTimer
vmirtDeleteModelTimer
vmirtSetModelTimer
vmirtClearModelTimer
vmirtIsModelTimerEnabled
vmirtGetModelTimerCurrentCount
vmirtGetModelTimerExpiryCount
```

opProcessorCycleCount
opProcessorICount

For related OP functions, see section 3.3.1.

2.5.2 Simulated Time

These functions handle interaction with simulated time:

```
vmirtGetQuantumStartTime
vmirtGetQuantumEndTime
vmirtGetLocalTime
vmirtGetMonotonicTime
vmirtGetQuantumStartTicks
vmirtGetQuantumEndTicks
vmirtGetLocalTicks
vmirtGetMonotonicTicks
```

opProcessorTime
opProcessorTicks

```
vmirtCreateQuantumTimer  
vmirtDeleteQuantumTimer
```

For related OP functions, see section 3.3.2.

2.5.3 Delay Estimation

These functions handle delay estimation:

<pre>vmirtSetDerateFactor vmirtGetDerateFactor vmirtAddSkipCount vmirtGetSkipCount</pre>	<pre>opProcessorDerate opProcessorSkipCyclesAdd opProcessorSkipCycles</pre>
--	--

For related OP functions, see section 3.3.3.

2.6 Processor Connections and Registers

2.6.1 Register Access

These functions implement access to processor registers:

<pre>vmirtGetRegGroupByName vmirtGetNextRegGroup vmirtGetRegByName vmirtGetNextReg vmirtGetNextRegInGroup vmirtRegRead vmirtRegWrite</pre>	<pre>opProcessorRegGroupByName opProcessorRegGroupNext opProcessorRegByName opProcessorRegNext opRegGroupRegNext opProcessorRegRead opProcessorRegWrite</pre>
--	---

For related OP functions, see section 3.9.1.

2.6.2 Bus Port Access

These functions implement access to processor bus ports:

```
vmirtGetBusPortByName  
vmirtGetNextBusPort
```

For related OP functions, see section 3.4.4.

2.6.3 Net Port Access

These functions implement access to processor net ports:

<pre>vmirtGetNetPortByName vmirtGetNextNetPort vmirtGetNetPortHandle vmirtWriteNetPort vmirtReadNetPort vmirtInstallNetCallback</pre>	<pre> opNetWrite opNetValue</pre>
---	---

For related OP functions, see section 3.4.15.

2.6.4 FIFO Port Access

These functions implement access to processor FIFO ports:

```
vmirtGetFifoPortByName  
vmirtGetNextFifoPort
```

For related OP functions, see section 3.4.8.

2.6.5 Exception Access

These functions implement access to processor exceptions:

```
vmirtGetCurrentException  
vmirtGetNextException
```

For related OP functions, see section 3.9.3.

2.6.6 Connection Objects

These functions are used to query and update connection objects:

```
vmirtConnGetInput  
vmirtConnGetOutput  
vmirtConnGetInputInfo  
vmirtConnGetOutputInfo  
vmirtConnGet  
vmirtConnPut  
vmirtConnNotifyGet  
vmirtConnNotifyPut
```

2.7 *Memory Operations*

2.7.1 Generic Load/Store

These functions implement load and store operations:

```
vmirtRead1ByteDomain  
vmirtRead2ByteDomain  
vmirtRead4ByteDomain  
vmirtRead8ByteDomain  
vmirtWrite1ByteDomain  
vmirtWrite2ByteDomain  
vmirtWrite4ByteDomain  
vmirtWrite8ByteDomain  
vmirtReadNByteDomain  
vmirtReadNByteDomainVA  
vmirtWriteNByteDomain  
vmirtWriteNByteDomainVA  
vmirtGetReadNByteSrc  
vmirtGetWriteNByteDst  
vmirtGetString
```

opMemoryRead

opMemoryWrite

For related OP functions, see section 3.5.1.

2.7.2 Memory Callbacks

These functions handle installation and removal of callback functions on memory accesses:

<code>vmirtAddReadCallback</code>	<code>opMemoryReadMonitorAdd</code>
<code>vmirtRemoveReadCallback</code>	<code>opMemoryReadMonitorDelete</code>
<code>vmirtAddWriteCallback</code>	<code>opMemoryWriteMonitorAdd</code>
<code>vmirtRemoveWriteCallback</code>	<code>opMemoryWriteMonitorDelete</code>
<code>vmirtAddFetchCallback</code>	<code>opMemoryFetchMonitorAdd</code>
<code>vmirtRemoveFetchCallback</code>	<code>opMemoryFetchMonitorDelete</code>

For related OP functions, see section 3.5.2.

2.7.3 Memory Manipulation

These functions are used to query and manipulate `memDomain` objects:

<code>vmirtGetProcessorCodeEndian</code>	
<code>vmirtGetProcessorDataEndian</code>	
<code>vmirtGetProcessorCodeDomain</code>	
<code>vmirtGetProcessorDataDomain</code>	
<code>vmirtSetProcessorCodeDomain</code>	
<code>vmirtSetProcessorDataDomain</code>	
<code>vmirtGetProcessorExternalCodeDomain</code>	
<code>vmirtGetProcessorExternalDataDomain</code>	
<code>vmirtGetProcessorInternalCodeDomain</code>	
<code>vmirtGetProcessorInternalDataDomain</code>	
<code>vmirtSetProcessorInternalCodeDomain</code>	
<code>vmirtSetProcessorInternalDataDomain</code>	
<code>vmirtSetProcessorCodeDomains</code>	
<code>vmirtSetProcessorDataDomains</code>	
<code>vmirtIsExecutable</code>	
<code>vmirtSetCreateDomainContext</code>	
<code>vmirtNewDomain</code>	
<code>vmirtGetDomainAddressBits</code>	
<code>vmirtGetDomainPrivileges</code>	
<code>vmirtGetDomainMapped</code>	
<code>vmirtGetNextMappedRange</code>	
<code>vmirtAliasMemory</code>	<code>opDynamicBridge</code>
<code>vmirtAliasMemoryPriv</code>	
<code>vmirtUnaliasMemory</code>	<code>opDynamicUnbridge</code>
<code>vmirtIsAlias</code>	
<code>vmirtMapVAToPA</code>	
<code>vmirtMapToDomain</code>	
<code>vmirtProtectMemory</code>	<code>opBusPrivSet</code>
<code>vmirtMapNativeMemory</code>	<code>opMemoryNativeDynamic</code>
<code>vmirtMapMemory</code>	
<code>vmirtMapCallbacks</code>	
<code>vmirtSetLoadStoreMask</code>	
<code>vmirtDebugDomain</code>	<code>opModuleDomainDebug</code>

For related OP functions, see section 3.5.3.

2.7.4 ASID Memory Management

These functions are used to implement ASID-based memory mappings:

```
vmirtSetProcessorASID
vmirtGetProcessorASID
vmirtAliasMemoryVM
vmirtUnaliasMemoryVM
vmirtGetDomainMappedASID
vmirtGetMRUStateTable
vmirtGetNthStateIndex
```

2.8 ***Floating Point***

2.8.1 **Floating-Point Operation Control**

These functions are used to control the behavior of floating-point operations:

```
vmirtSetSIMDMaxUnroll
vmirtConfigureFPU
vmirtGetFPControlWord
vmirtSetFPControlWord
vmirtRestoreFPState
```

2.8.2 **Floating-Point Operations**

These functions are used to implement floating-point operations:

```
vmirtGetFConvertRRDesc
vmirtGetFUnopRRDesc
vmirtGetFBinopRRRDesc
vmirtGetFTernopRRRRDesc
vmirtGetFCompareRRDesc
vmirtGetFCompareRRCDesc
vmirtFConvertSimdRR
vmirtFUnopSimdRR
vmirtFBinopSimdRRR
vmirtFTernopSimdRRRR
vmirtFCompareSimdRR
```

2.9 ***SMP Processor Hierarchy***

These functions are used to traverse SMP processor hierarchy:

vmirtGetSMPParent	opProcessorParent
vmirtSetSMPParent	
vmirtGetSMPChild	opProcessorChild
vmirtGetSMPPrevSibling	opProcessorSiblingPrevious
vmirtGetSMPNextSibling	opProcessorSiblingNext
vmirtGetSMPActiveSibling	
vmirtGetSMPCpuType	
vmirtGetSMPIndex	opProcessorIndex
vmirtSetSMPIndex	
vmirtIterAllChildren	opProcessorIterChildren
vmirtIterAllDescendants	opProcessorIterDescendants
vmirtIterAllProcessors	opProcessorIterAll

For related OP functions, see section 3.6.

2.10 ***Object File Access***

For related OP functions, see section 3.7.

2.10.1 Translation between Object File Addresses and Names

These functions handle translation between object file addresses and names:

```
vmirtAddressLookup  
vmirtSymbolLookup
```

2.10.2 Add and Access Object Files

These functions are used to add and access object files:

```
vmirtAddSymbolFile  
vmirtNextSymbolFile  
vmirtGetSymbolFileName
```

2.10.3 Query Object File Symbols

These functions are used to query symbol information in object files:

```
vmirtGetSymbolByName  
vmirtGetSymbolByNameFile  
vmirtGetSymbolByAddr  
vmirtGetSymbolByAddrFile  
vmirtNextSymbolByName  
vmirtNextSymbolByAddr  
vmirtPrevSymbolByAddr  
vmirtNextSymbolByNameFile  
vmirtNextSymbolByAddrFile  
vmirtPrevSymbolByAddrFile  
vmirtGetSymbolName  
vmirtGetSymbolAddr  
vmirtGetSymbolLoadAddr  
vmirtGetSymbolType  
vmirtGetSymbolBinding  
vmirtGetSymbolSize
```

2.10.4 Query Object File File/Line

These functions are used to query file/line information in object files:

```
vmirtGetFLByAddr  
vmirtGetFLByAddrFile  
vmirtNextFLByAddr  
vmirtPrevFLByAddr  
vmirtNextFLByAddrFile  
vmirtPrevFLByAddrFile  
vmirtGetFLFileName  
vmirtGetFLLineNumber  
vmirtGetFLAddr
```

2.11 *Range Table Hash*

These functions are used to implement address range hash tables:

```
vmirtNewRangeTable  
vmirtFreeRangeTable  
vmirtInsertRangeEntry  
vmirtRemoveRangeEntry  
vmirtGetFirstRangeEntry  
vmirtGetNextRangeEntry
```

```
vmirtGetRangeEntryLow  
vmirtGetRangeEntryHigh  
vmirtGetRangeEntryUserData  
vmirtSetRangeEntryUserData
```

2.12 Shared Data

These functions are used to access shared data:

vmirtFindSharedData	opSharedDataFind
vmirtFindAddSharedData	opSharedDataFindAdd
vmirtFindProcessorSharedData	
vmirtFindAddProcessorSharedData	
vmirtRemoveSharedData	opSharedDataDelete
vmirtSetSharedDataValue	opSharedDataValueSet
vmirtGetSharedDataValue	opSharedDataValueSet
vmirtRegisterListener	opSharedDataListenerRegister
vmirtUnregisterListener	opSharedDataListenerUnregister
vmirtWriteListeners	opSharedDataListenersWrite

For related OP functions, see section 3.8.

2.13 Notifiers

2.13.1 Register Update Notifiers

These functions are used to handle notifiers when registers are written:

```
vmirtAddRegisterWatchCallback  
vmirtDeleteRegisterWatchCallback
```

2.13.2 Branch Reason Notifiers

These functions are used to handle notifiers when branch events occur:

```
vmirtRegisterBranchNotifier  
vmirtUnregisterBranchNotifier
```

2.14 Save/Restore Support

These functions implement save and restore of processor state:

vmirtSave	opStateItemSave
vmirtRestore	opStateItemRestore
vmirtSaveElement	
vmirtRestoreElement	
vmirtSaveModelTimer	
vmirtRestoreModelTimer	
vmirtSaveDomain	opMemoryStateSave
vmirtRestoreDomain	opMemoryStateRestore
vmirtGetPostSlotCB	
vmirtSetPostSlotCB	

For related OP functions, see section 3.11.

2.15 Debug View Support

These functions implement debug view objects:

```
vmirtGetProcessorViewObject  
vmirtSetViewObjectUserData  
vmirtGetViewObjectUserData  
vmirtAddViewObject  
vmirtSetViewObjectConstValue  
vmirtSetViewObjectRefValue  
vmirtSetViewObjectValueCallback  
vmirtAddViewAction  
vmirtAddViewEvent  
vmirtNextViewEvent  
vmirtTriggerViewEvent  
vmirtDeleteViewObject
```

2.16 *Instruction Attributes*

These are instruction attributes interface functions (some in `vmiInstructionAttrs.h`):

```
vmiiaGetAttrs                opProcessorInstructionAttributes  
vmiiaConvertRegInfo          opRegConvert  
vmirtRegImplRaw  
vmirtEvaluateCondition
```

For related OP functions, see section 3.12.

2.17 *Shared Object / Dynamic Linked Library Loading*

These functions implement loading of shared objects:

```
vmirtDLOpen  
vmirtDLError  
vmirtDLSymbol  
vmirtDLClose
```

2.18 *Command Interpreter*

These functions implement access to the standard command interpreter:

```
vmirtAddCommand  
vmirtAddCommandParse  
vmirtAddArg  
vmirtAddArgEnum  
vmirtFindArgValue
```

For related OP functions, see section 3.13.

2.19 *Debugger Integration*

These functions implement integration with debug:

```
vmirtGetProcessorScope  
vmirtEvaluateGDBExpression  
vmirtEvaluateCodeLocation  
vmirtDisassemble                opProcessorDisassemble  
vmirtDisassembleInstr          opProcessorDisassembleInstruction  
vmirtInstructionBytes           opProcessorInstructionBytes
```

For related OP functions, see section 3.14.

2.20 *Trace Integration*

These functions implement integration with trace:

<code>vmirtTraceOnAfter</code>	<code>opProcessorTraceOnAfter</code>
<code>vmirtTraceOffAfter</code>	<code>opProcessorTraceOffAfter</code>

For related OP functions, see section 3.18.

2.21 *Documentation*

These functions implement processor documentation (in `vmiDoc.h`):

<code>vmidocAddSection</code>	<code>opDocSectionAdd</code>
<code>vmidocAddText</code>	<code>opDocTextAdd</code>
<code>vmidocProcessor</code>	
<code>vmidocAddFields</code>	
<code>vmidocAddField</code>	
<code>vmidocAddConstField</code>	

For related OP functions, see section 3.19.

2.22 *Messages*

These functions implement messaging and output (in `vmiMessage.h`):

<code>vmiMessage</code>	<code>opMessage</code>
<code>vmiVMessage</code>	<code>opVMessage</code>
<code>vmiPrintf</code>	<code>opPrintf</code>
<code>vmiVPrintf</code>	<code>opVPrintf</code>
<code>vmiAbort</code>	

For related OP functions, see section 3.20.

2.23 *HTTP Interface*

This function implements the HTTP interface (in `vmiHTTP.h`):

<code>vmihttpOpen</code>	<code>opModuleHTTPOpen</code>
--------------------------	-------------------------------

For related OP functions, see section 3.21.

2.24 *Licensing*

These functions are used to implement licensing:

<code>vmirtGetLicense</code>	
<code>vmirtGetLicenseErrString</code>	

2.25 *Encapsulated Model*

These functions implement encapsulated model semihosting:

<code>vmirtEncapIntercept</code>	
----------------------------------	--

3 OP Functions

3.1 *Simulation*

3.1.1 Session Control

These functions control the simulation session:

```
opSessionAtExit  
opSessionBuildDate  
opSessionCancelTextRedirect  
opSessionDebuggerNotifiersAdd  
opSessionDestFnSet  
opSessionExit  
opSessionFeaturesSet  
opSessionInit  
opSessionProductName  
opSessionProductVersion  
opSessionTerminate  
opSessionTextRedirect
```

3.1.2 Execution Control

These functions control simulation flow:

```
opProcessorSimulate  
opRootModuleSetSimulationRandomSeed  
opRootModuleSetSimulationStopTime  
opRootModuleSetSimulationStopTicks  
opRootModuleSetSimulationTimePrecision  
opRootModuleGetSimulationTimePrecision  
opRootModuleSetSimulationTimeSlice  
opRootModuleSetWallClockFactor  
opRootModuleSimulate  
opRootModuleStopReason  
opRootModulePostElaborate  
opRootModulePostSimulate  
opRootModulePreSimulate  
opRootModuleSetDebugStopTime  
opRootModuleSetDebugStopTicks  
opRootModuleTimeAdvance  
opRootModuleTicksAdvance  
opObjectSimulatorPhase  
opObjectSimulatorPhaseString  
opStopReasonString
```

3.1.3 Environment Access

These functions are used to access simulation environment features:

```
opBanner  
opErrors  
opLicPersonalitySet  
opNoBanner
```

3.1.4 Module Simulation

These functions control execution of a module:

```
opModuleFinish  
opModuleFinishStatus
```

3.2 Program Counter and Code Dictionary

These functions handle access to simulated program counter and invalidation of code dictionary:

```
opMemoryFlush  
opMemoryNativeFlush  
opProcessorFlush  
opProcessorPC  
opProcessorPCDS  
opProcessorPCInDS  
opProcessorPCNext  
opProcessorPCSet  
  
vmirtGetPC  
vmirtGetPCDS  
  
vmirtSetPC
```

For related VMI functions, see section 2.3.

3.3 Time and Cycle Counts

3.3.1 Instruction/Cycle Counting and Interrupt

These functions handle instruction and cycle counts:

```
opProcessorClocks  
opProcessorCycleCount  
opProcessorICount  
  
vmirtGetICount  
vmirtGetExecutedICount
```

For related VMI functions, see section 2.5.1.

3.3.2 Simulated Time

These functions handle interaction with simulated time:

```
opProcessorClocksUntilTime  
opProcessorClocksUntilTicks  
opProcessorTime  
opProcessorTicks  
opEventTimeNext  
opEventTicksNext  
opModuleCurrentTime  
opModuleCurrentTicks  
  
vmirtGetLocalTime  
vmirtGetLocalTicks
```

For related VMI functions, see section 2.5.2.

3.3.3 Delay Estimation

These functions handle delay estimation:

```
opProcessorDelay  
opProcessorDelayAdd  
opProcessorDerate  
  
vmirtSetDerateFactor
```

```
opProcessorSkipCyclesAdd  
opProcessorSkipCycles
```

```
vmirtAddSkipCount  
vmirtGetSkipCount
```

For related VMI functions, see section 2.5.3.

3.4 Platform Component Creation, Connection and Query

3.4.1 Bus Bridges

These functions operate on bus bridges:

```
opBridgeBusConnect  
opBridgeNew  
opBridgeNext
```

3.4.2 Buses

These functions operate on buses:

```
opBusAddrBits  
opBusMappedRangeNext  
opBusMaxAddress  
opBusNew  
opBusNext  
opBusShow
```

3.4.3 Bus Slaves

These functions operate on bus slaves:

```
opBusSlaveAddrHi  
opBusSlaveAddrLo  
opBusSlaveNew  
opBusSlaveNext
```

3.4.4 Bus Ports

These functions operate on bus ports:

```
opBusPortAddrBitsDefault  
opBusPortAddrBitsMax  
opBusPortAddrBitsMin  
opBusPortAddrHi  
opBusPortDescription  
opBusPortDomainType  
opBusPortDomainTypeString  
opBusPortIsDynamic  
opBusPortMMRegisterNext  
opBusPortMustConnect  
opBusPortType  
opBusPortTypeString
```

For related VMI functions, see section 2.6.2.

3.4.5 Bus Port Connections

These functions operate on bus port connections:

```
opBusPortConnAddrHi
```

```
opBusPortConnAddrLo  
opBusPortConnBus  
opBusPortConnIsDynamic  
opBusPortConnMapNotify  
opBusPortConnNext  
opBusPortConnType  
opBusPortConnTypeString
```

3.4.6 Extensions

These functions operate on processor extensions:

```
opExtElabExtension  
opExtensionNew  
opExtensionPath
```

3.4.7 FIFOs

These functions operate on FIFOs:

```
opFIFODepth  
opFIFONew  
opFIFONext  
opFIFOShow
```

3.4.8 FIFO Ports

These functions operate on FIFO ports:

```
opFIFOPortDescription  
opFIFOPortMustConnect  
opFIFOPortType  
opFIFOPortTypeString  
opFIFOPortWidth
```

For related VMI functions, see section 2.6.4.

3.4.9 FIFO Port Connections

These functions operate on FIFO port connections:

```
opFIFOPortConnFIFO  
opFIFOPortConnNext  
opFIFOPortConnWidth
```

3.4.10 MMCs

These functions operate on memory model components:

```
opMMCBusConnect  
opMMCNew  
opMMCNext  
opMMCPath  
opMMCTransparent
```

3.4.11 Memory-Mapped Registers

These functions operate on memory-mapped registers:

```
opMMRegisterBits
```

opMMRegisterDescription
opMMRegisterFieldBits
opMMRegisterFieldDescription
opMMRegisterFieldNext
opMMRegisterFieldOffset
opMMRegisterFieldReadable
opMMRegisterFieldReset
opMMRegisterFieldWritable
opMMRegisterIndex
opMMRegisterIsVolatile
opMMRegisterName
opMMRegisterOffset
opMMRegisterReadable
opMMRegisterView
opMMRegisterWritable

3.4.12 Memories

These functions operate on memories:

opMemoryBusConnect
opMemoryMaxAddress
opMemoryNativeNew
opMemoryNew
opMemoryNext
opMemorySpecParse

3.4.13 Modules

These functions operate on modules:

opFixedModuleNew
opModuleBusShow
opModuleNew
opModuleNewFromAttrs
opModuleNext
opModuleObject
opModulePath
opModulePurpose
opModuleShow
opRoot
opRootModuleDelete
opRootModuleNew

3.4.14 Nets

These functions operate on nets:

opNetNew
opNetNext
opNetShow
opNetValue
opNetValuePrevious
opNetWrite
opNetMonitorNext
opNetWriteMonitorAdd

3.4.15 Net Ports

These functions operate on net ports:

```
opNetPortDescription  
opNetPortMustConnect  
opNetPortType  
opNetPortTypeString
```

For related VMI functions, see section 2.6.3.

3.4.16 Net Port Connections

These functions operate on net port connections:

```
opNetPortConnNet  
opNetPortConnNext  
opNetPortConnType
```

3.4.17 Objects

These functions operate on generic objects:

```
opObjectBusPortConnNext  
opObjectBusPortNext  
opObjectByName  
opObjectClass  
opObjectClassSet  
opObjectExtElabNext  
opObjectExtensionNext  
opObjectFIFOPortConnNext  
opObjectFIFOPortNext  
opObjectHierName  
opObjectModule  
opObjectName  
opObjectNetConnect  
opObjectNetPortConnNext  
opObjectNetPortNext  
opObjectPacketnetPortConnNext  
opObjectPacketnetPortNext  
opObjectParent  
opObjectReleaseStatus  
opObjectReleaseStatusString  
opObjectRootModule  
opObjectType  
opObjectVLNV  
opObjectVisibility  
opObjectVisibilityString  
opVoidParent  
opVoidByName
```

3.4.18 Packet Nets

These functions operate on packet nets:

```
opPacketnetMaxBytes  
opPacketnetNew  
opPacketnetNext  
opPacketnetShow  
opPacketnetWrite  
opPacketnetMonitorNext  
opPacketnetWriteMonitorAdd
```

3.4.19 Packet Net Port Connections

These functions operate on packet net port connections:

```
opPacketnetPortConnNext
opPacketnetPortConnPacketnet
opPacketnetPortDescription
opPacketnetPortMustConnect
```

3.4.20 Peripherals

These functions operate on peripherals:

```
opPeripheralBusConnectMaster
opPeripheralBusConnectSlave
opPeripheralBusConnectSlaveDynamic
opPeripheralExtensionNew
opPeripheralFIFOConnect
opPeripheralNew
opPeripheralNext
opPeripheralPacketnetConnect
opPeripheralPath
opPeripheralSerialNotify
opPeripheralStopReason
```

3.4.21 Processors

These functions operate on processors:

```
opProcessorAMP
opProcessorAlternateName
opProcessorBusConnect
opProcessorBusConnectMaster
opProcessorBusConnectSlave
opProcessorDefaultSemihost
opProcessorDescription
opProcessorElfCodes
opProcessorEndian
opProcessorExceptionCurrent
opProcessorExceptionNext
opProcessorExtensionNew
opProcessorFIFOConnect
opProcessorFamily
opProcessorFaultAddress
opProcessorGroupH
opProcessorGroupL
opProcessorHelper
opProcessorLoadPhysical
opProcessorModeCurrent
opProcessorModeNext
opProcessorNew
opProcessorNewFromAttrs
opProcessorNewWithSemihost
opProcessorNext
opProcessorPath
opProcessorQLQualified
opProcessorStopReason
opProcessorCurrent
opProcessorVariant
opProcessorExit
opProcessorFinish
opProcessorFreeze

vmirtGetCurrentMode
vmirtGetNextMode

vmirtGetCurrentProcessor
vmirtProcessorVariant
vmirtExit
vmirtFinish
```

opProcessorFrozen	
opProcessorUnfreeze	
opProcessorYield	vmirtYieldControl
opInterrupt	vmirtInterrupt
opInterruptRSP	vmirtStop

For related VMI functions, see section 2.2.

3.4.22 VLNV

These functions operate on VLNV entries:

```
opVLNVIter
opVLNVLibrary
opVLNVName
opVLNVNew
opVLNVOld
opVLNVString
opVLNVVendor
opVLNVVersion
```

3.5 *Memory Operations*

3.5.1 Generic Load/Store

These functions implement load and store operations:

opBusRead	
opBusWrite	
opMemoryRead	vmirtReadNByteDomain
opMemoryWrite	vmirtWriteNByteDomain
opProcessorRead	
opProcessorReadAbort	
opProcessorWrite	
opProcessorWriteAbort	

For related VMI functions, see section 2.7.1.

3.5.2 Memory Callbacks

These functions handle installation and removal of callback functions on memory accesses:

opBusFetchMonitorAdd	
opBusFetchMonitorDelete	
opBusReadMonitorAdd	
opBusReadMonitorDelete	
opBusWriteMonitorAdd	
opBusWriteMonitorDelete	
opMemoryFetchMonitorAdd	vmirtAddFetchCallback
opMemoryFetchMonitorDelete	vmirtRemoveFetchCallback
opMemoryReadMonitorAdd	vmirtAddReadCallback
opMemoryReadMonitorDelete	vmirtRemoveReadCallback
opMemoryWriteMonitorAdd	vmirtAddWriteCallback
opMemoryWriteMonitorDelete	vmirtRemoveWriteCallback
opProcessorFetchMonitorAdd	
opProcessorFetchMonitorDelete	
opProcessorReadMonitorAdd	
opProcessorReadMonitorDelete	

```
opProcessorWriteMonitorAdd  
opProcessorWriteMonitorDelete
```

For related VMI functions, see section 2.7.2.

3.5.3 Memory Manipulation

These functions are used to query and manipulate `memDomain` objects:

```
opDynamicBridge          vmirtAliasMemory  
opDynamicUnbridge        vmirtUnaliasMemory  
opMemoryNativeDynamic    vmirtMapNativeMemory  
opBusPrivSet             vmirtProtectMemory  
opBusSlavePriv  
opMemoryPriv  
opModuleDomainDebug      vmirtDebugDomain
```

For related VMI functions, see section 2.7.3.

3.6 SMP Processor Hierarchy

These functions are used to traverse SMP processor hierarchy:

```
opProcessorChild          vmirtGetSMPChild  
opProcessorIndex          vmirtGetSMPIndex  
opProcessorIsLeaf  
opProcessorIterAll        vmirtIterAllProcessors  
opProcessorIterChildren   vmirtIterAllChildren  
opProcessorIterDescendants  vmirtIterAllDescendants  
opProcessorParent         vmirtGetSMPParent  
opProcessorSiblingNext    vmirtGetSMPNextSibling  
opProcessorSiblingPrevious vmirtGetSMPPrevSibling
```

For related VMI functions, see section 2.9.

3.7 Application File Access

These functions are used to access application files:

```
opApplicationControls  
opApplicationElfCode  
opApplicationEndian  
opApplicationEntry  
opApplicationHeaderRead  
opApplicationLoaderInstall  
opApplicationOffset  
opApplicationPath  
opBusApplicationLoad  
opMemoryApplicationLoad  
opObjectApplicationNext  
opProcessorApplicationLoad  
opProcessorApplicationRead  
opProcessorApplicationSymbolAdd
```

For related VMI functions, see section 2.10.

3.8 Shared Data

These functions are used to access shared data:

<code>opSharedDataDelete</code>	<code>vmirtRemoveSharedData</code>
<code>opSharedDataFind</code>	<code>vmirtFindSharedData</code>
<code>opSharedDataFindAdd</code>	<code>vmirtFindAddSharedData</code>
<code>opSharedDataListenerRegister</code>	<code>vmirtRegisterListener</code>
<code>opSharedDataListenerUnregister</code>	<code>vmirtUnregisterListener</code>
<code>opSharedDataListenersWrite</code>	<code>vmirtWriteListeners</code>
<code>opSharedDataValueGet</code>	<code>vmirtSetSharedDataValue</code>
<code>opSharedDataValueSet</code>	<code>vmirtSetSharedDataValue</code>

For related VMI functions, see section 2.12.

3.9 Processor Registers, Exceptions and Modes

3.9.1 Processor Registers

These functions are used to access processor registers:

<code>opProcessorRegByIndex</code>	
<code>opProcessorRegByName</code>	<code>vmirtGetRegByName</code>
<code>opProcessorRegByUsage</code>	
<code>opProcessorRegDump</code>	
<code>opProcessorRegGroupByName</code>	<code>vmirtGetRegGroupByName</code>
<code>opProcessorRegGroupNext</code>	<code>vmirtGetNextRegGroup</code>
<code>opProcessorRegIsExtension</code>	
<code>opProcessorRegNext</code>	<code>vmirtGetNextReg</code>
<code>opProcessorRegNextGPacket</code>	<code>vmirtGetNextReg</code>
<code>opProcessorRegNextPPacket</code>	<code>vmirtGetNextReg</code>
<code>opProcessorRegRead</code>	<code>vmirtRegRead</code>
<code>opProcessorRegReadByName</code>	
<code>opProcessorRegWrite</code>	<code>vmirtRegRead</code>
<code>opProcessorRegWriteByName</code>	
<code>opRegAccessEnum</code>	
<code>opRegAccessString</code>	
<code>opRegBits</code>	
<code>opRegDescription</code>	
<code>opRegGroup</code>	
<code>opRegGroupName</code>	
<code>opRegGroupRegNext</code>	<code>vmirtGetNextRegInGroup</code>
<code>opRegIndex</code>	
<code>opRegIsAlias</code>	
<code>opRegName</code>	
<code>opRegReadOnly</code>	
<code>opRegUsageEnum</code>	
<code>opRegUsageString</code>	

For related VMI functions, see section 2.6.1.

3.9.2 Mode Access

These functions implement access to processor modes:

<code>opModeCode</code>
<code>opModeDescription</code>
<code>opModeName</code>

3.9.3 Exception Access

These functions implement access to processor exceptions:

```
opExceptionCode  
opExceptionDescription  
opExceptionName
```

For related VMI functions, see section 2.6.5.

3.10 Parameters

These functions implement object parameters.

3.10.1 Formal Parameters

```
opFormalBoolDefaultValue  
opFormalDescription  
opFormalEnumDefault  
opFormalEnumDescription  
opFormalEnumNext  
opFormalEnumValue  
opFormalGroup  
opFormalGroupDescription  
opFormalGroupName  
opFormalInt32Limits  
opFormalInt64Limits  
opFormalStringDefaultValue  
opFormalStringMaxLength  
opFormalSystem  
opFormalType  
opFormalTypeString  
opFormalUns32Limits  
opFormalUns64Limits  
opFormalValueOrigin  
opFormalValueOriginString  
opFormaldoubleLimits  
opModuleFormalsShow  
opObjectFormalGroupNext  
opObjectFormalNext
```

3.10.2 Actual Parameters

```
opObjectParamBoolValue  
opObjectParamDoubleValue  
opObjectParamEndianValue  
opObjectParamEnumValue  
opObjectParamInt32Value  
opObjectParamInt64Value  
opObjectParamNext  
opObjectParamPtrValue  
opObjectParamStringValue  
opObjectParamUns32Value  
opObjectParamUns64Value  
opParamBoolOverride  
opParamBoolSet  
opParamDoubleOverride  
opParamDoubleSet  
opParamEndianOverride  
opParamEndianSet  
opParamEnumOverride  
opParamEnumSet
```

```
opParamInt32Override
opParamInt32Set
opParamInt64Override
opParamInt64Set
opParamListFromArray
opParamPtrOverride
opParamPtrSet
opParamPtrValue
opParamStringOverride
opParamStringSet
opParamType
opParamTypeString
opParamUns32Override
opParamUns32Set
opParamUns64Override
opParamUns64Set
```

3.11 **Save/Restore Support**

These functions implement save and restore of processor state:

```
opFIFOStateRestore
opFIFOStateRestoreFile
opFIFOStateSave
opFIFOStateSaveFile
opMMCStateRestore
opMMCStateRestoreFile
opMMCStateSave
opMMCStateSaveFile
opMemoryStateRestore          vmirtRestoreDomain
opMemoryStateRestoreFile
opMemoryStateSave             vmirtSaveDomain
opMemoryStateSaveFile
opNetStateRestore
opNetStateRestoreFile
opNetStateSave
opNetStateSaveFile
opObjectSaveRestoreSupported
opProcessorStateRestore
opProcessorStateRestoreFile
opProcessorStateSave
opProcessorStateSaveFile
opRootModuleStateRestore
opRootModuleStateRestoreFile
opRootModuleStateSave
opRootModuleStateSaveFile
opStateItemRestore            vmirtRestore
opStateItemSave               vmirtSave
```

For related VMI functions, see section 2.14.

3.12 **Instruction Attributes**

These are instruction attributes interface functions:

```
opProcessorInstructionAttributes  vmiaaGetAttrs
opRegConvert                     vmiaaConvertRegInfo
```

For related VMI functions, see section 2.16.

3.13 *Command Interpreter*

These functions implement access to the standard command interpreter:

```
opCmdArgUsed  
opCmdDefaultApplication  
opCmdErrorHandler  
opCmdParseArgs  
opCmdParseFile  
opCmdParseStd  
opCmdParserAdd  
opCmdParserDelete  
opCmdParserNew  
opCmdParserReplace  
opCmdUsageMessage  
opCommandArgDescription  
opCommandArgIterAll  
opCommandArgName  
opCommandArgType  
opCommandArgTypeString  
opCommandCall  
opCommandCallByName  
opCommandHelp  
opCommandIterAll  
opCommandStringCall  
opModuleCommandsShow  
opObjectCommandNext  
opProcessorCommandIterAll
```

For related VMI functions, see section 2.18.

3.14 *Debugger Integration*

These functions implement integration with debuggers such as gdb:

```
opProcessorDebug  
opProcessorDebugHelper  
opProcessorDisassemble          vmirtDisassemble  
opProcessorGdbFlags  
opProcessorGdbPath  
opProcessorInstructionBytes      vmirtInstructionBytes
```

For related VMI functions, see section 2.19.

3.15 *Breakpoints*

These functions implement support for breakpoints:

```
opProcessorBreakpointAddrClear  
opProcessorBreakpointAddrSet  
opProcessorBreakpointICountClear  
opProcessorBreakpointICountSet
```

3.16 *Watchpoints*

These functions implement support for watchpoints:

```
opBusAccessWatchpointNew  
opBusReadWatchpointNew
```

```
opBusWriteWatchpointNew
opMemoryAccessWatchpointNew
opMemoryReadWatchpointNew
opMemoryWriteWatchpointNew
opProcessorAccessWatchpointNew
opProcessorExceptionWatchpointNew
opProcessorModeWatchpointNew
opProcessorReadWatchpointNew
opProcessorRegWatchpointNew
opProcessorWriteWatchpointNew
opRootModuleWatchpointNext
opWatchpointAddressHi
opWatchpointAddressLo
opWatchpointDelete
opWatchpointReg
opWatchpointRegCurrentValue
opWatchpointRegPreviousValue
opWatchpointReset
opWatchpointTriggeredBy
opWatchpointType
opWatchpointUserData
```

3.17 *Triggers*

These functions implement triggers on module and stop reason events:

```
opModuleTriggerAdd
opModuleTriggerAddTicks
opModuleTriggerDelete
opProcessorStopHandlerAdd
opProcessorStopHandlerDelete
```

3.18 *Trace Integration*

These functions implement integration with trace:

```
opPrintfTrace
opProcessorTraceBufferDisable
opProcessorTraceBufferDump
opProcessorTraceBufferEnable
opProcessorTraceHighPCSet
opProcessorTraceLowPCSet
opProcessorTraceOffAfter      vmirtTraceOffAfter
opProcessorTraceOnAfter       vmirtTraceOnAfter
```

For related VMI functions, see section 2.20.

3.19 *Documentation*

These functions implement documentation generation:

```
opDocChildNext
opDocFieldOffset
opDocFieldWidth
opDocIsTitle
opDocNodeType
opDocSectionAdd      vmidocAddSection
opDocText
opDocTextAdd         vmidocAddText
opModuleDocSectionAdd
opObjectDocNodeNext
```

For related VMI functions, see section 2.21.

3.20 *Messages*

These functions implement messaging and output:

<code>opLastMessage</code>	
<code>opMessage</code>	<code>vmiMessage</code>
<code>opMessageDisable</code>	
<code>opMessageEnable</code>	
<code>opMessageQuiet</code>	
<code>opMessageSetNoWarn</code>	
<code>opMessageSetQuiet</code>	
<code>opMessageVerbose</code>	
<code>opModuleDiagnosticLevelSet</code>	
<code>opPeripheralDiagnosticLevelSet</code>	
<code>opPrintf</code>	<code>vmiPrintf</code>
<code>opResetErrors</code>	
<code>opSprintf</code>	
<code>opVAbort</code>	
<code>opVMessage</code>	<code>vmiVMessage</code>
<code>opVPrintf</code>	<code>vmiVPrintf</code>
<code>opVSprintf</code>	

For related VMI functions, see section 2.22.

3.21 *HTTP Interface*

This function implements the HTTP interface:

<code>opModuleHTTPOpen</code>	<code>vmihttpOpen</code>
-------------------------------	--------------------------

For related VMI functions, see section 2.23.